conveniences for you. For example, the rspec-rails library can tell from your specs which controller you're testing.[1]

All of these variants work just fine with metadata, which you used in *Tag Filtering*, on page 25 to tag examples with extra information:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe WeatherStation, 'radar updates', uses_network: true do
end
```

We'll talk more about metadata in *Slicing and Dicing Specs with Metadata*.

it

Within an example group, it creates one example. You pass a description of the behavior you're specifying. As with describe, you can also pass custom metadata to help RSpec run specific examples differently. Here's how you might indicate the specs for a computer-controlled lawn sprinkler need access to a serial bus:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe Sprinkler do
  it 'waters the garden', uses_serial_bus: true
end
```

These two building blocks are enough to get a great start with BDD. But a crucial part of BDD is "getting the words right." Some concepts don't fit well into phrases with describe and it. Fortunately, RSpec provides different ways to word your specs.

Other Ways to Get the Words Right

Using describe makes the most sense when the examples in a group are all describing a single class, method, or module. Not everything fits into such a pat template of "subject does action," though.

context Instead of describe

Sometimes, you want to group examples together because they're related to a shared situation or condition. You could do so using describe:

07-structuring-code-examples/01/getting_the_words_right.rb
```
RSpec.describe 'A kettle of water' do
  describe 'when boiling' do
    it 'can make tea'
    it 'can make coffee'
  end
end
```

1. https://relishapp.com/rspec/rspec-rails/v/3-6/docs/controller-specs